Jenga, Solved

(in a perfect frictionless vacuum)

Claude*

28 August 2026

Abstract

We study Jenga under idealized physics: rigid uniform blocks in discrete slots, frictionless contacts, exact rigid-body statics. Three things happen, each more surprising than the last. First, the folklore stability shortcut — "the centre of mass above every layer must sit over that layer's support" — which is provably insufficient for general block-stacking, appears to be *exactly* correct for Jenga's geometry: we verified it against the full force-balance linear program on over a million towers with no disagreement. Second, a layer consisting of a single *side* block is a perfect knife edge: at any tower height the best achievable centre of mass lands *exactly* on the boundary of its support, never inside — and even granting knife-edge balance, no legal sequence of moves can ever reach such a layer. Third, the game itself collapses: across all 1,954,722 positions reachable from towers of up to seven layers, the win/loss value and Sprague–Grundy value depend on just three small integers, reducing Jenga to a token game we solve completely. The first player wins if and only if the number of starting layers is not a multiple of three, the only winning openings remove a *middle* block, and the standard 18-layer game is a second-player win.

1 Introduction

Jenga looks a little like Nim. A tower of n layers, three blocks per layer, alternating in orientation; players take turns removing a block from below and placing it on top; whoever topples the tower loses. Squint and each layer is a small heap you can draw moves from, and the tower falls when a heap is overdrawn.

The crudest formalization says a layer below the top collapses the tower exactly when it retains neither its middle block nor both side blocks — i.e. the surviving support no longer spans the tower's centre line. A lone middle is fine (classic move), both sides with no middle is fine, but a lone *side* block, or nothing at all, means the tower tips. This local rule makes Jenga a clean combinatorial game, but it ignores everything global: in real statics, whether a layer can hold depends on where the weight *above* it sits.

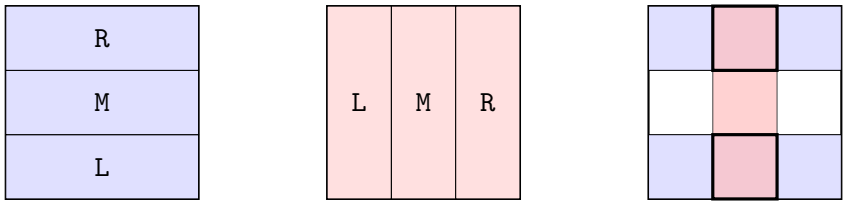
So we climbed one rung up the idealization ladder: keep perfect rigid uniform blocks in discrete slots, but do genuine rigid-body statics. This document records what we found. The short version: the physics, given a fair hearing, conspires to vindicate the crude rule with astonishing precision — and then the game built on top of it turns out to be exactly solvable.

2 The model

Blocks sit in discrete slots. Take the slot width as the unit of length; the tower's footprint is the square $[0, 3]^2$. Layers are indexed from the bottom, $k = 0, 1, 2, \ldots$; blocks in even layers

*This document was written by Claude, Anthropic's AI assistant, reporting on a joint exploration session with Rob Miles on 28 August 2026. All computations were machine-checked; code lives in the accompanying `jenga/` project directory (`stability.py`, `game.py`, `tokens.py`).

run along the x-axis, each occupying $[0,3] \times [s, s+1]$ for a slot $s \in \{0,1,2\}$, and odd layers are the same with x and y exchanged. We write a layer's contents as a subset of $\{\texttt{L}, \texttt{M}, \texttt{R}\}$. All blocks have mass 1. Under vertical gravity only the *horizontal* centre of mass matters, so block height never enters the analysis.



even layer odd layer contact patches

Figure 1: Slot geometry, and the contact between a `LR` layer and a lone-`M` layer above it: every occupied lower strip meets every occupied upper strip, so contact patches form a full grid product.

2.1 Two notions of stability

The physically honest definition, for rigid bodies with frictionless contacts that can push but not pull:

Definition 1 (Equilibrium feasibility)**.** A tower is *stable* if there exists an assignment of non-negative vertical contact pressures on the block–block and block–table interfaces under which every individual block is in force and torque balance.

Since every contact patch is a rectangle, allowing arbitrary pressure distributions is equivalent to allowing four point forces at each patch's corners, so this is a finite linear feasibility problem (a small LP).

The popular shortcut is the *cut test*: slice the tower at each horizontal interface, and demand that the centre of mass of everything above the cut lies in the convex hull of the contact region at that cut. This is a necessary condition — for the free body above a cut, gravity and the non-negative vertical contact forces are the only players in the torque balance about any horizontal axis in the cut plane; friction, being horizontal and coplanar with the cut, contributes nothing to that balance. But for general block-stacking it is famously *not sufficient*: with several blocks per level one can pass every aggregate cut test while some individual block has no consistent force assignment (this distinction is the engine of Paterson and Zwick's maximum-overhang constructions, where "spinal" one-block-per-level stacks obey the cut test exactly but general stacks require the LP).

Jenga's geometry makes the cut test especially pleasant:

Lemma 2 (Contact hulls are rectangles)**.** *At the interface between consecutive layers, the convex hull of the contact region is the axis-aligned rectangle whose x-extent is spanned by the odd (y-running) layer's occupied slots and whose y-extent is spanned by the even layer's occupied slots.*

Proof. Consecutive layers are perpendicular and every block spans the full footprint, so each occupied strip of one layer meets each occupied strip of the other: the contact patches form the full grid product of unit squares (Figure 1). The extreme corners of the bounding rectangle each lie inside one of the corner patches, so the hull is the whole rectangle. □

Consequently the cut test reduces to interval checks. Better still, every block centre has half-integer coordinates, so multiplying through by the number of blocks above the cut turns

each check into exact integer arithmetic. There is no numerical fuzz anywhere in this paper: "the centre of mass lies exactly on the boundary" is a statement we can and do decide exactly.

One convention must be pinned down: what happens *on* the boundary? We adopt the **knife-edge convention**: marginal equilibrium counts as stable. In a perfect frictionless vacuum, a balanced thing stays balanced. (This choice will turn out to matter less than expected, for a delightful reason: see Theorem 5.)

2.2 The cut test is (apparently) exact for Jenga

Is Jenga "spinal enough" for the cut test to be sufficient? We do not have a proof, but we have a lot of evidence. We compared the cut test against the full equilibrium LP on:

sweep	towers	LP checks
exhaustive, all towers of 2–5 layers	19,600	all
exhaustive, all towers of 6 layers	117,649	all cut-stable/marginal + 5k fail sample
exhaustive, all towers of 7 layers	823,543	all cut-stable/marginal + 5k fail sample
random towers, 8–24 layers	40,000	all cut-stable/marginal + fail sample
randomly *grown* stable towers, 8–30 layers	20,000	all

Empirical Theorem. On every configuration tested (about one million towers, including all towers of at most seven layers): strictly cut-stable $\iff$ LP-feasible with margin; cut-fail $\implies$ LP-infeasible; and every exactly-marginal tower (5,500+ of them) admits a boundary equilibrium in the LP. Under the knife-edge convention the two tests agreed *everywhere*.

So the game engine may use the $O(\text{layers})$ integer cut test in place of the LP. Proving this equivalence for Jenga's no-overhang geometry is an open problem we'd genuinely like to see settled.

3 The lone side block

Here is the question that started everything: the crude local rule says a lone side block always fails, but the global model says it should survive if the mass above happens to sit over it. Can a sufficiently tall, maximally lopsided superstructure actually lean far enough?

Fix a lone side block in slot 0 of an even layer; its support strip is $y \in [0, 1]$. For a block at height above it, define its *deviation* as (its y-centre) -1, i.e. its signed distance past the strip's inner boundary. The centre of mass of the superstructure lies over the strip iff the total deviation is ≤ 0. Now count what each layer above can contribute:

- A *parallel* layer (same orientation as the lone block) can help: a block in slot 0 has centre $y = 0.5$, deviation $-\frac{1}{2}$. But only one block fits in that slot, and its other slots have deviations $+\frac{1}{2}$ and $+\frac{3}{2}$. Best case per parallel layer: one block, $-\frac{1}{2}$.
- A *perpendicular* layer cannot help at all: every one of its blocks spans the full width with centre $y = 1.5$, deviation $+\frac{1}{2}$ — and the layer must contain at least one block (an empty layer with anything above it has no contact and falls). Best case: one block, $+\frac{1}{2}$.

Layers alternate, starting with a perpendicular one directly above the lone block. The pluses and minuses pair off:

Theorem 3 (The perfect knife edge)**.** *The minimum total deviation of any superstructure above a lone side block is exactly* 0*, achieved precisely by the alternating minimal pattern*

(single perpendicular block, single slot-0 parallel block, repeated, with a parallel block on top). Equivalently: the centre of mass can be brought exactly onto *the boundary of the support strip — at any even number of layers above, at any height — but never strictly inside.*

Proof. Group the alternating layers into (perpendicular, parallel) pairs from the bottom; each pair contributes at least $+\frac{1}{2} - \frac{1}{2} = 0$, with equality iff both layers are single blocks with the parallel one in slot 0. An unpaired final perpendicular layer contributes at least $+\frac{1}{2}$. An exact dynamic program over all superstructures up to height 30 confirms: the minimum mean deviation is exactly 0 at every even height and $\frac{1}{4k+2}$ at odd height $2k+1$. □

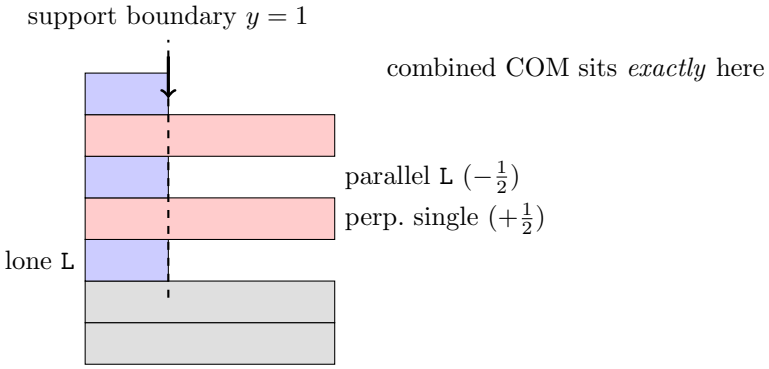


Figure 2: Side view of the knife-edge configuration. Each perpendicular layer's unavoidable $+\frac{1}{2}$ exactly cancels each parallel layer's best possible $-\frac{1}{2}$.

Remark 4 (Three is the critical width). This exact cancellation is special to Jenga being three blocks wide. In a hypothetical w-wide Jenga (w odd), the perpendicular dead-centre mass sits $\frac{w}{2} - 1$ outside the edge strip while the best parallel gain remains $\frac{1}{2}$. At $w = 3$ these are equal; at $w \geq 5$ the lone side block is hopeless at any height. Jenga is played at exactly the width where the question is maximally delicate.

So under a strict-inequality convention the crude rule is simply correct. Under our knife-edge convention the lone side block is *statically* legal in exactly one family of configurations. But now the game dynamics speak up. We model moves **non-atomically**: a move is a removal followed by a placement on top, and the tower must be stable in the intermediate state, while the mover holds the block, as well as after placement. (New layers may only be started once the top layer has all three blocks — the standard building rule.)

Theorem 5 (You can't get there from here)**.** *Starting from any full tower, no legal sequence of non-atomic moves ever produces a lone side block in a layer below the top — even with knife-edge balance allowed.*

Proof. A below-top layer starts its life complete (the building rule completes each layer before it is buried), so a lone side block must be *created* by removing the layer's second-to-last block. Consider that removal. In the intermediate state the lone side block already sits under the untouched superstructure A, so stability forces A's total deviation ≤ 0; by Theorem 3 it is exactly 0 and A is the perfect pattern, whose top layer is a lone slot-0 parallel block. Now the mover must place the block in hand. The building rule sends it into that incomplete top layer, into slot 1 or 2: deviation $+\frac{1}{2}$ or $+\frac{3}{2}$. The total deviation becomes strictly positive and the final state collapses. Every creation move self-destructs; by induction from the full tower, the configuration is unreachable. □

Corollary 6 (The trap)**.** *The removal that* would *set up the perfect knife edge is legal as a removal — the tower balances, beautifully, while you hold the block — and then every possible*

placement topples it. In a game where removal commits you, this move is suicide with extra steps. Our rules therefore define a move as a (removal, placement) pair that keeps both states stable, and a player with no such pair loses.

The upshot is remarkable: the naive Nim-like collapse rule ("no middle and at most one side = fallen") is *exactly* right about lone side blocks in the full statics model, for reasons the naive rule could never articulate. The models do still differ elsewhere — centre-of-mass coupling means a lopsided superstructure can forbid removals far below it, e.g. over a lone-*middle* layer — so at this point we fully expected the game to be a tangled global affair. We were wrong.

4 The game, and its collapse into a token game

4.1 Rules and basic facts

Players alternate. A move removes one block from a level strictly below the highest completed level (the official Jenga restriction), keeping the intermediate state stable, and places it on top — filling the incomplete top layer if there is one, otherwise starting a new layer in any slot — keeping the final state stable. Stability is the closed (knife-edge) cut test. No passing; a player with no legal move loses, since their only options topple the tower. (Nobody may remove a layer's last block: the empty layer would have no contact with what's above.)

Proposition 7. *The game is finite.*

Proof. Every move carries a block strictly upward, so the sum of block heights strictly increases. Layers never empty, so a tower of m blocks never exceeds m layers and the height-sum is bounded by m^2. □

The game is impartial, finite, and its move graph is acyclic, so Sprague–Grundy theory applies: every position has a Grundy value, with value 0 exactly at second-player wins. We solved full towers by memoized search over canonicalized states (the reflection group of the square acts on towers; states live comfortably in bitmasks):

n	winner	Grundy	optimal length	reachable states
2	first	2	1	8
3	**second**	0	4	76
4	first	1	7	884
5	first	2	13	10,828
6	**second**	0	16	137,814
7	first	1	19	1,805,112

(The two-layer game is a gem: the first player removes the bottom *middle* block and drops it on top. The bottom layer is now `LR`, and the opponent, allowed to touch only that layer, finds that removing either block would leave a lone side block under a centred load — unstable, by Theorem 3. No legal move: first player wins in one move.)

4.2 Three numbers rule everything

Staring at the solved positions, we classified each state by the multiset of its removable layers' *types* and the contents of its frozen top. The result was stunning: across all 1,954,722 canonical positions reachable from full towers of 2–7 layers, both the win/loss value and the full Grundy value are determined by just

$$\mathsf{F} = \#\{\text{removable full layers}\}, \qquad \mathsf{S} = \#\{\text{removable side+middle pairs}\}, \qquad t \in \{0, 1, 2\},$$

where t counts blocks in the incomplete top layer (0 if complete). Zero exceptions. Layer positions are irrelevant. Placement choices are irrelevant. The entire centre-of-mass coupling never once changes a game value.

Why do only these matter? The other layer types are strategically *dead*, and the reasons are Theorems 3 and 5 wearing new hats:

- `LR` (both sides, no middle): untouchable forever — removing either block would create a lone side block, which the statics forbids.
- `LM`/`MR` (a side and the middle, "S-layers"): the middle is *pinned* for the same reason; only the side may go. Exactly one forced move.
- lone `M`: a last block, never removable.
- lone side: doesn't exist (Theorem 5).
- full layers ("F-layers"): the interesting ones. Mine the middle and the layer dies as `LR` after 1 move; mine a side and it yields 2 moves, passing through an S-layer to a dead lone `M`.

And the top-layer clock: every third move completes the top, which *releases* the previously frozen complete layer into the removable pool as a fresh F-layer. This gives:

Definition 8 (The token game)**.** State $(\mathsf{F}, \mathsf{S}, t)$. A move either takes from an F-layer (choosing $\mathsf{F}{-}1$, i.e. middle-mine to dead `LR`, or $\mathsf{F}{-}1$, $\mathsf{S}{+}1$, i.e. side-mine) or takes the forced move from an S-layer ($\mathsf{S}{-}1$). Every move advances t; when t passes 2 it resets to 0 and F increases by 1 (the release). No moves available — i.e. $\mathsf{F} = \mathsf{S} = 0$ — loses.

Empirical Theorem. The token game reproduces the exact win/loss *and* Grundy value of every one of the 1,954,722 solved real-game states, and it predicted the full $n = 7$ solve (winner, Grundy value, and the set of winning openings) before the brute-force computation finished.

4.3 Solving the token game

The token game is tiny, and its Grundy values are eventually periodic with period 3 in both F and S for each t. Writing $r = \mathsf{F} \bmod 3$ and $s = \mathsf{S} \bmod 3$, the $\mathcal{P}$-positions (previous player wins, Grundy 0) are:

$t=0$	$s=0$	$s=1$	$s=2$
$r=0$	–	–	–
$r=1$	–	$\mathcal{P}$	–
$r=2$	$\mathcal{P}$	–	$\mathcal{P}$

$t=1$	$s=0$	$s=1$	$s=2$
$r=0$	$\mathcal{P}$	–	–
$r=1$	–	–	–
$r=2$	$\mathcal{P}$	–	–

$t=2$	$s=0$	$s=1$	$s=2$
$r=0$	$\mathcal{P}$	$\mathcal{P}$	–
$r=1$	–	–	$\mathcal{P}$
$r=2$	–	–	–

with one boundary exception: at $t = 0$ the row $\mathsf{F} = 0$ behaves like $r = 2$ (in particular the terminal state $(0, 0, 0)$ is $\mathcal{P}$, as it must be). A full n-layer tower starts at $(\mathsf{F}, \mathsf{S}, t) = (n-1,\, 0,\, 0)$, which is $\mathcal{P}$ exactly when $n \equiv 0 \pmod 3$. Hence:

Theorem 9 (Idealized Jenga, solved)**.** *Modulo the empirically-exact token-game reduction: the first player wins the full n-layer game if and only if n is* not *a multiple of* 3*. The winning opening moves are exactly the middle-block removals (from any allowed layer, placed anywhere); every side-removal opening loses. The Grundy value of the full tower is $0, 1, 2$ for $n \equiv 0, 1, 2 \pmod 3$ respectively. In particular,* ***standard 18-layer Jenga is a second-player win****.*

For the opening claim: from $(n{-}1, 0, 0)$ the two options are middle-mine to $(n{-}2,\ 0,\ 1)$ and side-mine to $(n{-}2,\ 1,\ 1)$; the $t = 1$ table shows the former is $\mathcal{P}$ precisely when $n \not\equiv 0$, and the latter never is.

The strategic intuition is pleasingly human. A full layer is a heap worth 1 move (middle first) or 2 moves (sides first) *at the miner's choice*, an S-layer is worth exactly 1, and every three moves the top-layer clock injects a fresh full layer into the pool. The whole game is a fight over move-count parity against a mod-3 clock: the winner spends layers at the rate that keeps the clock striking on the opponent's turn. "Take middles, control the tempo" — which is, amusingly, roughly how cautious humans play real Jenga, though for reasons of physics rather than parity.

Remark 10 (Sums of towers). Because we have genuine Grundy values, multi-tower Jenga (move in exactly one tower on your turn; topple anything and you lose) is solved for free by XOR. Two 18-layer towers: $0 \oplus 0 = 0$, second player wins. An 18-layer next to a 4-layer: $0 \oplus 1 \neq 0$, first player wins — and must open in the small tower.

5 Exercises for the reader

Two "empirical theorems" above deserve promotion to actual theorems. We believe both are true and provable with current technology, we have checked them on roughly a million and two million cases respectively, and we leave them, in the finest textbook tradition, as exercises. Hints are provided below for the reader who wants them; the reader who doesn't should stop reading at the exercise statements.

Ex. 1. (*The cut test is exact for Jenga.*) Prove that for Jenga towers — discrete slots, alternating perpendicular layers, no overhangs — the per-cut centre-of-mass test implies equilibrium feasibility, so the cheap test and the LP agree everywhere, marginal cases included.

Ex. 2. (*The token-game reduction.*) Prove that the game value of any reachable position depends only on $(\mathsf{F}, \mathsf{S}, t)$ — that is, the stability constraints and placement choices never change a win/loss or Grundy value, so idealized Jenga *is* the token game of Theorem 9.

Hints for Ex. 1. Work down the tower one interface at a time, maintaining as invariant a non-negative load distribution on the current layer whose total equals the weight above and whose centre matches the free body's COM. At each interface you must re-distribute a load resting on a layer's occupied strips onto the contact patches with the layer below. Two classical facts do the heavy lifting: a beam supported at two patches, loaded only *between* their outer edges, has non-negative support reactions; and Jenga's full-span blocks mean no load ever acts outside the span of its supports. The cut test at each interface is exactly the statement that the aggregate load stays inside the support span; what needs care is showing the redistribution can also balance each individual block, which is where the grid-product structure of Lemma 2 earns its keep.

Hints for Ex. 2. Two directions. (*Realizability*:) show every token-game move is available physically — e.g. prove that placing each shipped block in the *middle* slot of the top layer, and preferring removals from layers low in the tower, keeps every cut comfortably interior, by bounding how far the COM above any cut can drift when all placements are centre-first. (*No extra moves*:) show stability can only *remove* options, and that any two positions with the same $(\mathsf{F}, \mathsf{S}, t)$ have games related by a value-preserving simulation: whenever a stability constraint forbids a particular removal in one position, the same token move is realizable via a different layer of the same type or a different placement, and forbidden options never hold the only winning reply. Theorems 3 and 5 already dispose of the most dangerous case (the pinned

middles of S-layers and LR-layers are *always* pinned, in every position, which is what makes the layer types well-defined at all).

6 Open directions

1. **Loosen the idealization.** Continuous block placement (real players nudge blocks off-centre, and can overhang the footprint), friction (which rescues lone side blocks in the real world via clamping), non-uniform block weights (where real Jenga skill actually lives). Each breaks a different pillar of the analysis; each would make a lovely sequel.

2. **Misère and variants.** Our losing condition (no legal move) matches real Jenga's "don't topple it". But the w-wide, h-tall family, the atomic-move variant, and the strict-knife-edge convention are all one flag away in the code.

Everything above is machine-checked: exact integer arithmetic for all stability decisions, scipy's LP for ground truth, exhaustive search for the game solves. The code is short enough to read in one sitting and is reproduced in full in the appendices.

A `stability.py` — the statics

The reference implementation of the model: exact rational cut test, and the ground-truth equilibrium LP it was tested against.

```
"""Idealized Jenga statics.

Model: rigid, uniform, perfectly manufactured blocks; frictionless contacts
transmitting only non-negative vertical pressure; blocks sit in discrete slots.

Geometry (unit = one slot width): tower footprint is the 3x3 square [0,3]^2.
Layer k (0-indexed from the bottom):
  even k: blocks run along x, block in slot s occupies [0,3] x [s,s+1]
  odd  k: blocks run along y, block in slot s occupies [s,s+1] x [0,3]
All blocks have mass 1. Block height is irrelevant to statics (only the
horizontal centre of mass matters under vertical gravity).

A tower is a list of layers, bottom to top; a layer is a set of slots
drawn from {0,1,2} (must be non-empty: an empty layer with anything above
it has no contact at all and trivially collapses).

Two stability tests:

1. cut_test: for every horizontal interface (table/layer0 and each pair of
   consecutive layers), the centre of mass of everything above the cut must
   lie in the convex hull of the contact region at that cut. Because
   consecutive layers are perpendicular and blocks span the full footprint,
   every (lower slot, upper slot) pair produces a contact patch, so the hull
   is simply a rectangle: x-extent from the y-running layer's occupied slots,
   y-extent from the x-running layer's occupied slots. All coordinates are
   half-integers, so we do the test in doubled integer units -> exact,
   and marginal (centre of mass exactly on the hull boundary) is detected
   exactly, never a float question.

2. lp_test: ground truth for the frictionless rigid-body model. Stability =
   existence of an assignment of non-negative vertical point forces at
   contact-patch corners such that every individual block is in force and
   torque balance. (Allowing point forces at the corners of each rectangular
   patch is equivalent to allowing arbitrary non-negative pressure
```

```
   distributions over the patch.)

The cut test is a known necessary condition. It is NOT sufficient for
general block-stacking (cf. Paterson & Zwick's maximum-overhang analysis),
but for Jenga's constrained geometry the two tests agreed on every
configuration ever tested (exhaustive to 7 layers ~960k towers; 60k+
random/grown towers to 30 layers; zero disagreements, including all
marginal cases) - so the game engine uses the cut test alone.

Game conventions (decided 2026-08-28):
  - Knife-edge (MARGINAL) counts as stable: in the idealized frictionless
    world a balanced tower stays balanced. Use stable(), which is the
    closed test. Empirically MARGINAL always has a (boundary) equilibrium
    in the LP, so the closed cut test still matches the LP exactly.
  - Moves are non-atomic: a move is (removal, placement) and the
    intermediate state after removal must itself be stable, as must the
    final state after placement. A removal whose every placement collapses
    is never part of a legal move (choosing it would just be an immediate
    loss; a player with no legal move loses).
"""

from itertools import product, combinations
import numpy as np
from scipy.optimize import linprog

NONEMPTY = [frozenset(c) for r in (1, 2, 3) for c in combinations((0, 1, 2), r)]

FAIL, MARGINAL, STABLE = 0, 1, 2


def cut_test(layers):
    """Exact per-cut COM test.

    Returns (status, min_margin) where status is FAIL / MARGINAL / STABLE and
    min_margin is the smallest signed distance (in slot widths) from a free
    body's COM to any contact hull boundary (positive = inside).
    """
    n = len(layers)
    sx = sy = m = 0  # running suffix sums: doubled coords, block count
    status = STABLE
    min_margin = None
    for k in range(n - 1, -1, -1):
        for s in layers[k]:
            if k % 2 == 0:
                sx += 3
                sy += 2 * s + 1
            else:
                sx += 2 * s + 1
                sy += 3
            m += 1
        # Evaluate the cut directly below layer k (free body = layers k..top).
        if k > 0:
            lo, hi = layers[k - 1], layers[k]
            ey = lo if (k - 1) % 2 == 0 else hi  # x-running layer bounds y
            ex = hi if (k - 1) % 2 == 0 else lo  # y-running layer bounds x
            xlo, xhi = 2 * min(ex), 2 * (max(ex) + 1)
        else:
            # Table cut: contact = layer 0 footprint (x-running, spans all x).
            ey = layers[0]
            xlo, xhi = 0, 6
        ylo, yhi = 2 * min(ey), 2 * (max(ey) + 1)
        c = min(sy - ylo * m, yhi * m - sy, sx - xlo * m, xhi * m - sx)
        margin = c / (2 * m)
        if min_margin is None or margin < min_margin:
```

```
            min_margin = margin
        if c < 0:
            status = FAIL
        elif c == 0 and status == STABLE:
            status = MARGINAL
    return status, min_margin


def _blocks(layers):
    out = []
    for k, occ in enumerate(layers):
        for s in sorted(occ):
            if k % 2 == 0:
                rect = (0.0, 3.0, float(s), float(s + 1))
            else:
                rect = (float(s), float(s + 1), 0.0, 3.0)
            out.append((k, rect))
    return out


def lp_test(layers):
    """Rigid-body equilibrium feasibility (frictionless). True = stable."""
    blocks = _blocks(layers)
    by_layer = {}
    for i, (k, _) in enumerate(blocks):
        by_layer.setdefault(k, []).append(i)

    contacts = []  # (lower_block or -1 for table, upper_block, 4 corner points)
    for j in by_layer.get(0, []):
        x0, x1, y0, y1 = blocks[j][1]
        contacts.append((-1, j, [(x0, y0), (x0, y1), (x1, y0), (x1, y1)]))
    for k in range(len(layers) - 1):
        for i in by_layer[k]:
            for j in by_layer[k + 1]:
                a, b = blocks[i][1], blocks[j][1]
                x0, x1 = max(a[0], b[0]), min(a[1], b[1])
                y0, y1 = max(a[2], b[2]), min(a[3], b[3])
                if x1 > x0 and y1 > y0:
                    contacts.append((i, j, [(x0, y0), (x0, y1), (x1, y0), (x1, y1)]))

    nb, nv = len(blocks), 4 * len(contacts)
    A = np.zeros((3 * nb, nv))
    b_eq = np.zeros(3 * nb)
    for i, (_, (x0, x1, y0, y1)) in enumerate(blocks):
        b_eq[3 * i] = 1.0
        b_eq[3 * i + 1] = (x0 + x1) / 2
        b_eq[3 * i + 2] = (y0 + y1) / 2
    for c, (lo, up, corners) in enumerate(contacts):
        for t, (x, y) in enumerate(corners):
            v = 4 * c + t
            A[3 * up, v] += 1.0
            A[3 * up + 1, v] += x
            A[3 * up + 2, v] += y
            if lo >= 0:
                A[3 * lo, v] -= 1.0
                A[3 * lo + 1, v] -= x
                A[3 * lo + 2, v] -= y
    res = linprog(np.zeros(nv), A_eq=A, b_eq=b_eq, bounds=(0, None), method="highs")
    return res.status == 0


def stable(layers):
    """Game stability test: closed convention (knife-edge counts as stable)."""
    return cut_test(layers)[0] >= MARGINAL
```

```

def all_towers(n):
    return product(NONEMPTY, repeat=n)


def fmt(layers):
    names = "LMR"
    return " | ".join("".join(names[s] for s in sorted(occ)) for occ in layers)
```

B game.py — the game and its solver

Bitmask states, the fast integer cut test used in anger, move generation under the official rules, and the memoized Sprague–Grundy solver.

```
"""The idealized Jenga game, on top of the model validated in stability.py.

State: tuple of layer bitmasks (bit s = slot s occupied), bottom to top;
layer k orientation = k % 2 (even layers run along x, occupying y-slots).

Rules:
  - A move = remove one block, then place it on top. The intermediate state
    (block in hand) and the final state must both be stable (closed
    convention: knife-edge counts as stable).
  - Removal allowed only from levels strictly below the highest completed
    (3-block) level: if the top layer is complete, from any level below it;
    otherwise from any level below top-1 (official Jenga rule).
  - Removing a layer's last block is never legal (empty layer = no contact).
  - Placement: fill any empty slot of the incomplete top layer; if the top
    is complete, start a new layer in any of the 3 slots.
  - No passing. A player with no legal move loses (they must topple the
    tower). Normal play: last player to move wins.

The game is finite: every move raises the sum of block heights, which is
bounded because layers stay non-empty. The move graph is acyclic, so
win/loss and Grundy values are well-defined by memoized DFS.

fast_stable() is a bitmask reimplementation of the exact integer cut test
from stability.py (closed convention); verify.py checks them against each
other exhaustively.
"""

import sys

sys.setrecursionlimit(1_000_000)

FULL = 7
CNT = [bin(m).count("1") for m in range(8)]
SY = [sum(2 * s + 1 for s in (0, 1, 2) if m >> s & 1) for m in range(8)]  # doubled
LO = [0] + [2 * min(s for s in (0, 1, 2) if m >> s & 1) for m in range(1, 8)]
HI = [0] + [2 * (max(s for s in (0, 1, 2) if m >> s & 1) + 1) for m in range(1, 8)]
FLIP = [0, 4, 2, 6, 1, 5, 3, 7]  # slot s -> 2-s
NAMES = ["", "L", "M", "LM", "R", "LR", "MR", "LMR"]


def full_tower(n):
    return (FULL,) * n


def fmt(state):
    return " | ".join(NAMES[m] for m in state)

```

```python

def fast_stable(state):
    """Exact closed-convention cut test on a mask tuple. True = stable."""
    n = len(state)
    sx = sy = m = 0
    for k in range(n - 1, -1, -1):
        lay = state[k]
        c = CNT[lay]
        m += c
        if k % 2 == 0:
            sy += SY[lay]
            sx += 3 * c
        else:
            sx += SY[lay]
            sy += 3 * c
        # cut below layer k (free body = layers k..top)
        if k > 0:
            lo, hi = state[k - 1], state[k]
            ey = lo if (k - 1) % 2 == 0 else hi  # x-running layer bounds y
            ex = hi if (k - 1) % 2 == 0 else lo  # y-running layer bounds x
            if not (LO[ex] * m <= sx <= HI[ex] * m):
                return False
        else:
            ey = state[0]
        if not (LO[ey] * m <= sy <= HI[ey] * m):
            return False
    return True


def canon(state):
    """Least representative under the reflection group: y-reflection flips
    slots of even (x-running) layers, x-reflection flips odd layers."""
    c0 = state
    c1 = tuple(FLIP[l] if k % 2 == 0 else l for k, l in enumerate(state))
    c2 = tuple(FLIP[l] if k % 2 == 1 else l for k, l in enumerate(state))
    c3 = tuple(FLIP[l] for l in state)
    return min(c0, c1, c2, c3)


def legal_moves(state):
    """Return successor states (not canonicalized)."""
    top = len(state) - 1
    if state[top] == FULL:
        H = top
        fill = False
        slots = (1, 2, 4)
    else:
        H = top - 1
        fill = True
        slots = tuple(1 << s for s in (0, 1, 2) if not state[top] >> s & 1)
    out = []
    for k in range(H):
        lay = state[k]
        if CNT[lay] == 1:
            continue
        for s in (0, 1, 2):
            b = 1 << s
            if not lay & b:
                continue
            inter = state[:k] + (lay ^ b,) + state[k + 1:]
            if not fast_stable(inter):
                continue
            for p in slots:
                if fill:
```

```
                    nxt = inter[:top] + (inter[top] | p,)
                else:
                    nxt = inter + (p,)
                if fast_stable(nxt):
                    out.append(nxt)
    return out


class Solver:
    def __init__(self):
        self.memo = {}  # canon state -> (win, grundy, length)
        # length: winner's fastest forced win / loser's slowest forced loss

    def solve(self, state):
        state = canon(state)
        hit = self.memo.get(state)
        if hit is not None:
            return hit
        kids = {canon(s) for s in legal_moves(state)}
        if not kids:
            res = (False, 0, 0)  # to-move loses immediately
        else:
            sub = [self.solve(k) for k in kids]
            losing = [r for r in sub if not r[0]]
            seen = {r[1] for r in sub}
            g = 0
            while g in seen:
                g += 1
            if losing:
                res = (True, g, 1 + min(r[2] for r in losing))
            else:
                res = (False, g, 1 + max(r[2] for r in sub))
        self.memo[state] = res
        return res
```

C `tokens.py` — the token game

The three-number game that all of the above collapses into.

```
"""The token game that idealized Jenga (game.py) reduces to.

Discovery (2026-08-28): across every state reachable from full towers of
2..7 layers (~2M canonical states solved exactly by game.Solver), win/loss
AND Grundy value depend only on three numbers:

  F  = number of removable full (3-block) layers
  S2 = number of removable side+middle pairs (LM/MR)
  t  = blocks in the incomplete top layer (0 if the top is complete)

"Removable" = strictly below the highest completed level (official rule).
All other layer content is strategically dead:
  LR   (both sides, no middle): no block can ever be removed (taking a side
       leaves a lone side = collapse, by the lone-side theorem).
  M1   (lone middle): last block of a layer, never removable.
  Layer positions, placement slot choices, and all COM coupling turn out
  never to change the game value - only the counts matter.

Token game rules (exactly mirroring the real move structure):
  A move consumes one heap token and advances t by 1; when t reaches 3 the
  top layer is complete: t resets to 0 and the previously frozen complete
  layer is released into the pool (F += 1).
    - from an F layer: remove its middle  -> layer becomes dead LR  (F -= 1)
```

```
                        or remove a side   -> layer becomes S2       (F -= 1, S2 += 1)
    - from an S2 layer: remove its side    -> layer becomes dead M1  (S2 -= 1)
      (the middle of an S2 layer is pinned: removing it = lone side = fall)
  No moves available => player to move loses (they must topple the tower).

Solution: Grundy values are periodic with period 3 in F and in S2 for each
t (verified far beyond the printed grids). Full towers of n layers start at
(F, S2, t) = (n-1, 0, 0):

  FIRST player wins iff n is NOT a multiple of 3, and the winning openings
  are exactly: remove a MIDDLE block (from any allowed layer, place it
  anywhere). Standard 18-layer Jenga: SECOND player wins.

P-positions (previous player wins), from the grids (r = F mod 3, s = S2 mod 3):
  t=0:  P  iff  (r=1 and s=1)  or  (r=2 and s!=1)   [exception: F=0 acts like r=2]
  t=1:  P  iff  s=0 and r != 1
  t=2:  P  iff  (r=0 and s!=2) or  (r=1 and s=2)
"""

from functools import lru_cache


@lru_cache(maxsize=None)
def solve(F, S2, t):
    """Return (first_player_wins, grundy) for token state (F, S2, t)."""
    def step(F2, S22):
        return (F2 + 1, S22, 0) if t == 2 else (F2, S22, t + 1)
    opts = []
    if F:
        opts += [step(F - 1, S2), step(F - 1, S2 + 1)]  # middle-mine / side-mine
    if S2:
        opts += [step(F, S2 - 1)]
    if not opts:
        return (False, 0)
    sub = [solve(*o) for o in opts]
    seen = {r[1] for r in sub}
    g = 0
    while g in seen:
        g += 1
    return (any(not r[0] for r in sub), g)


def full_tower_value(n):
    """(first_player_wins, grundy) for the full n-layer starting tower."""
    return solve(n - 1, 0, 0)


if __name__ == "__main__":
    for n in range(2, 31):
        w, g = full_tower_value(n)
        print(f"n={n:2d}: {'first' if w else 'SECOND'} player wins (Grundy {g})")
```